Class Project

Nutritional Analytics from Code-switched Gastronomy Influencer Reels Using Prompt Engineering and Large Language Models

PROJECT SPECIFICATION

BITP 3123 Distributed Application Development

Author

Emaliana Kasmuri

Fakulti Teknologi Maklumat dan Komunikasi

Universiti Teknikal Malaysia Melaka

Table of Contents

1 Document History

Date	Description
5 April 2026	Document creation.
18 June 2026	Change LLM size from large to small-to-medium size in Section 5.2 and Section 7.

2 Description

This project, titled **Nutritional Analytics from Code-switched Gastronomy Influencer Reels Using Prompt Engineering and Large Language Models**, develops a dashboard application for gastronomical scientists to explore the potential of artificial intelligence and large language models (LLMs) in automating nutritional analytics. The system extracts nutritional information from Malaysian gastronomy influencer Instagram Reels by transcribing cooking videos and applying prompt engineering techniques including zero-shot, few-shot, chain-of-thought, and structured output to guide LLMs in identifying ingredients and estimating their nutritional values from English–Malay code-switched transcripts. The dashboard enables researchers to run and compare experiments across multiple LLM and prompt technique combinations, annotate ground truth data, export evaluation results, and assess model performance through a comprehensive set of text similarity, numeric accuracy, and output quality metrics.

3 Instruction to the Students

You are required to develop a dashboard application named **MasakGramPrompt** for the project titled Nutritional Analytics from Code-switched Gastronomy Influencer Reels Using Prompt Engineering and Large Language Models. This document serves as a guide for the development of the MasakGramPrompt application.

4 Pipeline flow

A pipeline refers to a sequence of connected processing stages where the output of one stage becomes the input of the next, enabling structured and automated data flow from start to finish. As illustrated in Figure 1, the **MasakGramPrompt** pipeline operates across three stages. In the first stage, a cooking video transcript extracted from an Instagram Reel is combined with a prompt constructed using one of the four prompt engineering techniques to form the input to the LLM. In the second stage, the combined transcript and prompt are passed to the LLM for nutritional analysis, where the model identifies ingredients and estimates their corresponding nutritional values based on the instructions encoded in the prompt. In the final stage, the LLM produces a structured result in JSON format containing the extracted ingredient list and nutritional information, which is subsequently stored in the MySQL database for evaluation and export.

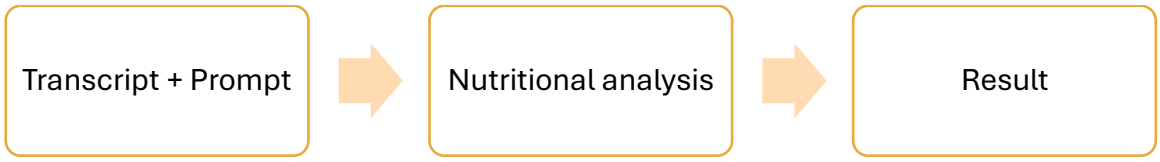

Figure 1: MasakGramPrompt pipeline flow

5 Project Specification

5.1 System Architecture

a. The system must be developed using either Java TCP/IP socket programming or Java Spring Framework.
b. The system must operate in a distributed environment where the client interface, application logic, and LLM inference service run as separate communicating components.
c. All components must follow full object-oriented programming (OOP) principles including encapsulation, inheritance, polymorphism, and abstraction.

5.2 LLM Model Selection

a. In a single execution (processing all transcript), the user must be able to select:-

i. One LLM models to execute for analysis from the following available models: **Llama 3.2 (3B)**, **Phi-4-mini (3.8B)**, **Qwen 2.5 (3B)**, **Gemma-SEA-LION v4 (4B)** and **MedGemma (4B)**.

ii. One or more prompt engineering techniques to apply: **zero-shot**, **few-shot**, **chain-of-thought**, and **structured output**.

b. The system must display the execution status of each selected model and technique combination in real time, indicating whether the condition is pending, running, or completed.

5.3 Reel Analysis Dashboard

a. The dashboard must display the following reel details for each selected Instagram Reel: reel ID, influencer handle, video duration, language tag (English, Malay, or code-switched), transcript status, and ground truth availability.

b. The dashboard must display a transcript preview for each reel, with code-switched Malay terms visually distinguished from English terms.

c. The dashboard must show the analysis status for each LLM model and prompt technique combination applied to the reel, presented in a clear and readable layout.

d. The dashboard must provide a direct link to the nutritional fact sheet for each reel.

5.4 Nutritional Fact Sheet

a. The nutritional fact sheet must display the ground truth ingredients as annotated by human annotators, serving as the reference baseline.

b. The nutritional fact sheet must display the ingredients extracted by each LLM model side by side with the ground truth for direct comparison.

c. Each ingredient entry must include the following fields: ingredient name (raw), ingredient name (English), quantity expression, quantity category, quantity unit, quantity value, language tag, and hallucination flag.

d. The nutritional fact sheet must display computed nutritional values per ingredient and recipe-level totals, including calories, protein, fat, carbohydrates, sodium, and other standard nutrition facts panel fields.

e. The nutritional fact sheet must be downloadable as a CSV file.

5.5 Data Export

a. The user must be able to select which data to download from the ten available export options described in Table 1, where each option produces one CSV file corresponding to a specific evaluation layer.
b. The ten CSV files to be exported, their corresponding SQL query labels in **metrics_evaluation_queries.sql**, and the metrics they feed into are described in Table 1.

Table 1: Data export files, SQL query labels, and their metric targets

Data	File Name	SQL Query Label
Exact match	layer1a_exact_match.csv	LAYER 1A
Text similarity	layer1b_text_similarity.csv	LAYER 1B
Numeric quantity	layer2a_numeric_quantity.csv	LAYER 2A
Numeric nutrition	layer2b_numeric_nutrition.csv	LAYER 2B
Nutrition totals	layer2c_nutrition_totals.csv	LAYER 2C
JSON validity	layer3a_json_validity.csv	LAYER 3A
Hallucination	layer3b_hallucination.csv	LAYER 3B
Ingredient detection	layer3c_ingredient_detection.csv	LAYER 3C
Human evaluation	layer4_human_evaluation.csv	LAYER 4
Condition scores	layer5_condition_scores.csv	LAYER 5

6 Prompt Design

The project employs four prompt engineering techniques as listed in **Error! Reference source not found.**. Each implemented as a pair of plain-text files stored in a folder named **prompts**, yielding eight files in total. The files are:-

a. zero_shot_system.txt

b. zero_shot_user.txt
c. few_shot_system.txt
d. few_shot_user.txt
e. chain_of_thought_system.txt
f. chain_of_thought_user.txt
g. structured_output_system.txt,
h. structured_output_user.txt.

The _**system** suffix denotes the system prompt, which establishes the model's role, behavioral rules, and technique-specific instructions, while the _**user** suffix denotes the user prompt, which supplies the actual cooking video transcript alongside a JSON scaffold for the model to populate. Table 2 summarizes the composition of each system and user prompt pair across the four techniques.

Table 2: Prompt design by technique

Technique	**System Prompt**	**User Prompt**
Zero-shot	Role + rules only	Transcript + JSON scaffold
Few-shot	Role + rules + study example instruction	Worked example (*telur goreng*) + transcript + JSON scaffold
Chain-of-thought	Role + rules + step-by-step reasoning instruction	Transcript + 7 explicit reasoning steps + JSON scaffold
Structured output	Role + full schema rules + unit conversion table + handling rules	Transcript + JSON scaffold only

Across all eight files**, {{TRANSCRIPT}}** serves as a consistent runtime placeholder that the Java pipeline replaces with the actual transcript text via a simple string substitution, and the JSON schema embedded in every scaffold is expanded to capture the full nutrition facts profile, including saturated fat, cholesterol, sodium, dietary fiber, total sugars, vitamin D, calcium, iron, and potassium.

7 Database Design

All data collected, processed, and generated throughout the MasakGramPrompt pipeline is persisted in a **MySQL** relational database, with the schema defined in **create_tables.sql** and the reference seed data supplied in **seed_data.sql**. The database comprises eleven tables whose design is fully documented **in BITP 3123 – Sem 2 2025/2026 – 03 Database Design.pdf**.

The first group of tables captures the source data: influencer stores profile information for each Malaysian gastronomy influencer, reel records metadata for each downloaded Instagram Reel linked to its influencer, audio_file tracks the extracted MP3 audio derived from each reel, and transcript holds the Faster-Whisper-generated text transcription along with its language tag reflecting English–Malay code-switching.

The second group supports ground truth annotation and experiment configuration. ground_truth_reel stores reel-level nutritional totals as annotated by human annotators, and ground_truth_ingredient records per-ingredient annotations using controlled vocabularies for quantity category, culinary unit, culinary value, and quantity expression. llm_model catalogs the **five** locally hosted Ollama models, which are Llama 3.2 (3B), Phi-4-mini (3.8B), Qwen 2.5 (3B), Gemma-SEA-LION v4 (4B) and MedGemma (4B). The prompt_technique registers the four prompt engineering techniques by storing relative file paths to their corresponding system and user prompt files rather than the prompt text itself.

The third group captures experiment outputs. experiment logs each of the sixteen experimental runs produced by crossing four models with four prompt techniques. nutrition_result stores the reel-level nutritional values extracted by the LLM for each run, and ingredient_result records the per-ingredient extraction output including the raw and normalized ingredient name, quantity expression, language tag, and hallucination flag.

Seed data for the four LLM models and four prompt techniques is preloaded via **seed_data.sql** to ensure consistent foreign key references before any experiment is executed.

End of Document